CIS 3300 · INHERITANCE AND POLYMORPHISM, PART 2 OF 4

Polymorphism and Casting

One variable, many kinds of objects, and how Java decides which method runs

---

Part 1 ended with a small surprise: code written inside `Employee` ran `Manager`'s version of `getMonthlyPay`. This set of notes explains why. It introduces the difference between a variable's declared type and an object's actual type, shows how that difference lets one array, one list, or one method handle every kind of employee, and then covers the tools for going back the other way: casting, `instanceof`, and `getClass`. For reading, see Core Java, Volume I, Chapter 5, Sections 5.1 and 5.2.

A note before you begin. These are my notes from over the years, cleaned up so they are readable. They are not a study guide, and they are not gospel: not everything here appears in a lecture, and not every topic has notes yet. I write from memory and revise every semester. I also use AI to help convert documents to LaTeX, check for issues, and add explanations. I do not trust it completely, so I am still editing, and a typo or imprecise statement can survive longer than it should. Java also changes over time, though the basics we cover are resilient. If you find something that looks wrong, contradicts the book, or does not behave the way I claim when you run it, tell me. You will be doing the other students a favor, and you will understand the material better for having caught it. The textbook is the tiebreaker, and the compiler beats us both. These notes sometimes go beyond the book, but they do not replace the assigned chapters or the recorded lectures, and neither of those replaces the notes. Read the chapter, watch the lecture, and study the notes.

1. What Polymorphism Means

The word means "many forms." You will often read that polymorphism is "the ability of an object to take on many forms." That phrasing is common, but it is misleading, because an object never changes. A `Manager` is created as a `Manager` and stays a `Manager` until it is garbage collected. What varies is how we *refer* to it. A more accurate definition:

> **Polymorphism** is the ability of a variable of a superclass type to refer to objects of many subclasses, with each object still responding to method calls in its own way.

The idea that makes it possible is called the *substitution principle*: an object of a subclass can be used anywhere an object of its superclass is expected. A `Manager` is an `Employee`, so a `Manager` can go in an `Employee` variable, an `Employee[]` array, an `ArrayList<Employee>`, or an `Employee` parameter.

```
Employee someone = new Manager("Grace", 90000, 12000);   // allowed
Manager boss = new Employee("Ada", 60000);               // does not compile
```

The first line is fine because every manager is an employee. The second fails because not every employee is a manager. Every Java object can pass at least two is-a tests (its own class and `Object`), so in this sense every Java object is polymorphic.

2. Declared Type and Actual Type

Every reference involves two types, and keeping them apart is the key to this whole unit.

Table 1: The two types behind every reference.

Name	What it is	Who uses it
Declared type (static type)	The type written in the variable's declaration	The compiler, to decide which method calls are *allowed*
Actual type (runtime type)	The class used with `new` when the object was created	The JVM, to decide which overridden method *runs*

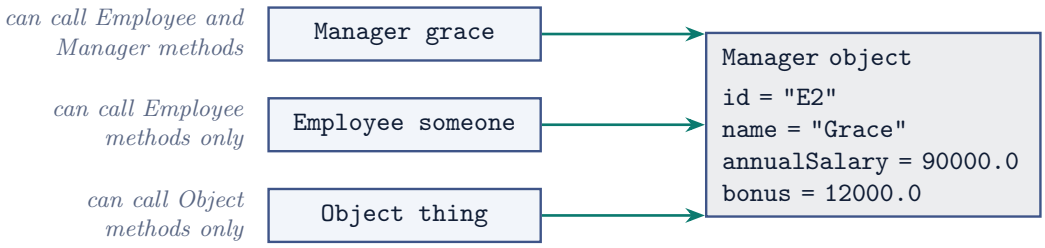


Figure 1: Three variables with three declared types, all referring to one object whose actual type is `Manager`. The declared type limits what you can *ask for*. The actual type decides what *happens*: `someone.getMonthlyPay()` runs `Manager`'s version, and `thing.toString()` runs the `toString` that `Manager` inherits or overrides.

```
Manager grace = new Manager("Grace", 90000, 12000);
Employee someone = grace;     // declared type Employee, actual type Manager
Object thing = grace;         // declared type Object, actual type Manager

System.out.println(someone.getMonthlyPay());   // 8500.0: Manager's version runs
someone.getBonus();                            // does not compile
```

```
error: cannot find symbol
  symbol:   method getBonus()
  location: variable someone of type Employee
```

The error message names the declared type. The compiler does not track which object a variable will hold at run time; it only knows that `someone` is an `Employee`, and `Employee` has no `getBonus`. Figure 1 summarizes the rule: the declared type decides what you are *allowed to call*, and the actual type decides *which version runs*.

3. Which Method Runs: Binding

Connecting a method call in the source code to the method body that executes is called *binding*. Java does part of this work at compile time and part at run time.

Dynamic binding: overriding, decided at run time

When you call an instance method that can be overridden, the JVM looks at the actual class of the object, finds the most specific version of that method, and runs it. This is *dynamic binding* (also

called late binding, or runtime polymorphism). It is why `Employee.toString` printed Grace's bonus-inclusive pay in Part 1: inside `toString`, the call `getMonthlyPay()` is really `this.getMonthlyPay()`, and `this` is a `Manager`.

Static binding: overloading, decided at compile time

Choosing among *overloads* happens earlier. The compiler picks the overload whose parameters best match the *declared* types of the arguments, and that choice is fixed in the compiled code. This is *static binding* (also called early binding, or compile-time polymorphism). Calls to `private`, `static`, and `final` methods, and to constructors, are also bound at compile time, because there is only one version they could ever mean.

This program has two overloads, one for `Employee` and one for `Manager`:

```
public static void printBadge(Employee e) {
    System.out.println("Badge: STAFF   " + e.getName());
}

public static void printBadge(Manager m) {
    System.out.println("Badge: MANAGER " + m.getName());
}
```

```
Manager grace = new Manager("Grace", 90000, 12000);
Employee someone = grace;     // the same object
printBadge(grace);
printBadge(someone);
```

```
Badge: MANAGER Grace
Badge: STAFF   Grace
```

The same object got two different badges. The compiler saw a `Manager` argument in the first call and an `Employee` argument in the second, and it chose the overloads before the program ever ran. If you want behavior to depend on the actual object, put it in an overridden method, not in overloads.

Table 2: Static binding compared with dynamic binding.

Question	Static binding	Dynamic binding
Also called	Early binding, compile-time polymorphism	Late binding, runtime polymorphism
Decided by	The compiler	The JVM, while the program runs
Based on	Declared types	The actual class of the object
Applies to	Overloaded methods; `private`, `static`, and `final` methods; constructors	Overridden instance methods

Is dynamic binding slow? You may read that dynamic binding is slower because the method is looked up while the program runs. In early Java that was a real consideration. Today the JVM optimizes these calls so well that the difference is almost never measurable in programs like ours. Choose the design that is clearest, not the one you think is faster.

Fields and static methods are not polymorphic

Only overridable instance methods use dynamic binding. If a subclass declares a field with the same name as a superclass field, the subclass object has *two* fields, and which one you get depends on the declared type. The same is true of `static` methods, which are hidden rather than overridden. Both situations are confusing enough that the advice is simple: do not reuse a superclass field name in a subclass, and call static methods through the class name, as the review notes required.

4. Polymorphism in Arrays, Lists, and Methods

This is where polymorphism pays for itself. One array or one list can hold every kind of employee, and one loop can process them all without knowing which kind each one is.

```
Employee ada = new Employee("Ada", 60000);
Manager grace = new Manager("Grace", 90000, 12000);
Executive alan = new Executive("Alan", 150000, 30000, 24000);

Employee[] team = { ada, grace, alan };
for (Employee e : team) {
    System.out.println(e);   // each object uses its own toString and getMonthlyPay
}
```

```
E1 Ada, monthly pay 5000.00
E2 Grace, monthly pay 8500.00, bonus 12000.00
E3 Alan, monthly pay 17000.00, bonus 30000.00, stock 24000.00
```

`Executive` extends `Manager`. Its `getMonthlyPay` calls `super.getMonthlyPay()`, which is `Manager`'s version, which in turn calls `Employee`'s version. Each class adds only its own piece.

A method parameter works the same way. This method was written once, knows only about `Employee`, and works for every subclass, including subclasses nobody has written yet:

```
public static double totalMonthlyPay(ArrayList<Employee> staff) {
    double total = 0;
    for (Employee e : staff) {
        total += e.getMonthlyPay();
    }
    return total;
}
```

That last point is the real payoff. When the company adds a new kind of employee next year, the payroll code above does not change at all. Only the new class is written.

A detail about lists. An `ArrayList<Employee>` can hold a `Manager`. However, an `ArrayList<Manager>` is *not* an `ArrayList<Employee>`, so you cannot pass one to `totalMonthlyPay` above. The reason is that the method could then add a plain `Employee` to a list that promised to hold only managers. Generics, in Week 13, show how to write methods that accept both.

5. Casting Between Types

Casting a reference changes the *declared* type you are using to look at an object. It never changes the object.

Table 3: Upcasting and downcasting.

Kind	Direction	Written how?	Can it fail?
Upcast	Subclass to superclass, such as `Manager` to `Employee`	Automatically; no cast needed	No; every manager is an employee
Downcast	Superclass to subclass, such as `Employee` to `Manager`	Explicitly: `(Manager) e`	Yes; checked at run time

Why you need a downcast

Polymorphism lets you store a manager in an `Employee` variable, but then the compiler only lets you call `Employee` methods. To reach `setBonus`, you need a reference whose declared type is `Manager`:

```
Employee e = staff.get(1);
Manager m = (Manager) e;          // "I know this employee is a manager"
m.setBonus(m.getBonus() + 1000);
```

The cast is a promise to the compiler. The JVM checks the promise when the line runs. If the object is not actually a `Manager` (or a subclass of `Manager`), it throws a `ClassCastException`:

```
Employee notAManager = new Employee("Ada", 60000);
Manager oops = (Manager) notAManager;   // compiles, then fails at run time
```

```
java.lang.ClassCastException: class payroll.Employee cannot be cast to class payroll.
    Manager
```

Casts the compiler rejects

If the compiler can prove that a cast could never succeed, it refuses to compile it. Casting between two classes where neither is a subclass of the other is one such case. In Part 3 we add `HourlyEmployee` as a sibling of `Manager`, and this happens:

```
Manager m = new Manager("Grace", 90000, 12000);
HourlyEmployee h = (HourlyEmployee) m;   // does not compile
```

```
error: incompatible types: Manager cannot be converted to HourlyEmployee
```

A manager can never be an hourly employee, so there is no way to "cast sideways" to a sibling. However, if the variable's declared type were `Employee`, the same cast would compile, because an `Employee` variable might hold an `HourlyEmployee`. It would then fail at run time with a `ClassCastException`.

6. Checking Before You Cast: instanceof

To avoid a `ClassCastException`, test the object first. The `instanceof` operator returns `true` if the object is an instance of the named class *or any of its subclasses*.

```
for (Employee e : staff) {
    if (e instanceof Manager) {
        Manager m = (Manager) e;
        m.setBonus(m.getBonus() + 1000);
    }
}
```

Since Java 16, `instanceof` can test and cast in one step. This is called *pattern matching*. The variable `m` is created only when the test succeeds:

```
for (Employee e : staff) {
    if (e instanceof Manager m) {        // test, cast, and name it m
        m.setBonus(m.getBonus() + 1000);
    }
}
```

With Ada, Grace, and Alan in the list, Grace and Alan both get the raise, because an `Executive` is also a `Manager`. Two more facts are worth remembering: `null instanceof Manager` is `false`, never an exception, and the compiler rejects an `instanceof` test that could never be true, just as it rejects an impossible cast.

instanceof or getClass

Both of these check an object's type while the program runs. They answer different questions.

```
Employee boss = new Executive("Alan", 150000, 30000, 24000);
System.out.println(boss instanceof Manager);           // true
System.out.println(boss.getClass() == Manager.class);  // false
System.out.println(boss.getClass().getSimpleName());   // Executive
```

Table 4: Comparing instanceof with getClass.

Question	`x instanceof T`	`x.getClass() == T.class`
What it is	An operator	A method inherited from `Object`, compared with `==`
Question it answers	Can `x` be used as a `T`?	Was `x` created as exactly a `T`?
Subclasses	Count as a match	Do not count as a match
When `x` is `null`	`false`	Throws `NullPointerException`
Typical use	Before a downcast; most type checks	An exact-class check, such as some `equals` methods

Use `instanceof` unless you have a specific reason to exclude subclasses. That fits the substitution principle: if an `Executive` is a `Manager`, code that works with managers should accept executives too.

equals in a hierarchy

The choice matters most inside `equals`. In our payroll program, two employees are the same employee when they have the same ID, and that rule does not change for managers or executives. When the rule is fixed in the superclass, Core Java recommends using `instanceof` and making `equals final`, so no subclass can break it:

```
@Override
public final boolean equals(Object other) {
    if (this == other) {
        return true;
    }
    if (!(other instanceof Employee)) {   // also false when other is null
        return false;
    }
    Employee that = (Employee) other;
    return id.equals(that.id);
}

@Override
public final int hashCode() {
    return id.hashCode();
}
```

If instead each subclass needs its own idea of equality (for example, two managers are equal only if their bonuses also match), use `getClass() != other.getClass()` in place of the `instanceof` test. Otherwise `employee.equals(manager)` and `manager.equals(employee)` could give different answers, which breaks the rule that `equals` must be symmetric.

A warning sign

If you catch yourself writing a long chain like this, stop:

```
if (e instanceof Executive x) { pay = ...; }
else if (e instanceof Manager m) { pay = ...; }
else { pay = ...; }
```

That decision belongs inside the classes, as an overridden method, so that each class answers for itself and new classes need no changes elsewhere. Use `instanceof` when you need a feature only one subclass has, such as `setBonus`, not to imitate overriding.

7. Summary

Table 5: Summary of the ideas in these notes.

Idea	Key point
Polymorphism	A superclass variable can refer to many kinds of subclass objects; each responds its own way
Substitution principle	A subclass object can be used wherever its superclass is expected
Declared type	Decides which calls compile
Actual type	Decides which overridden method runs
Dynamic binding	Overridden methods, chosen at run time by the object
Static binding	Overloads, chosen at compile time by declared types
Upcast	Automatic and always safe
Downcast	Explicit; checked at run time; may throw `ClassCastException`
`instanceof`	True for the class and its subclasses; false for `null`; can bind a variable
`getClass`	The exact class; subclasses do not match

8. Try It

1. Predict the output, then run it: create an `Executive`, store it in a variable of each type (`Executive`, `Manager`, `Employee`, `Object`), and call `toString()` through each one. Which calls give different results, and why?
2. Write a method `totalBonuses(Employee[] team)` that adds up the bonuses of only the managers and executives in an array. Which tool from these notes do you need, and why can you not just call `getBonus()` on every element?
3. In `Employee.equals`, replace the `instanceof` test with a `getClass` test. Put a `Manager` into an `ArrayList<Employee>`, then call `contains` with the same object stored in an `Employee` variable. Does the result change? Now explain what would happen if `equals` were not `final` and `Manager` overrode it.